

Seminar 7

Transformata Fourier Rapidă (FFT)

Evaluarea polinoamelor, înmulțirea rapidă
și algoritmul Cooley–Tukey

Asist. univ. dr. Mihai Tudor

30 martie 2026

1 Motivație și context

Problema: Fie un polinom $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ de grad $n - 1$. Vrem să evaluăm A în n puncte distincte simultan.

Metodă	Complexitate	Observație
Horner (un singur punct)	$\Theta(n)$	Optim pentru un punct
"Naivă" (n puncte)	$\Theta(n^2)$	Aplicăm Horner de n ori
FFT (n puncte speciale)	$\Theta(n \log n)$	Alegerea inteligentă a punctelor

Cheia FFT: alegem ca puncte de evaluare **rădăcinile complexe ale unității** de ordin n , care au proprietăți algebrice speciale ce permit descompunerea recursivă a problemei.

Observație 1.1. *FFT nu face evaluarea într-un singur punct mai rapidă decât $\Theta(n)$. Ceea ce face este să transforme evaluarea în n puncte dintr-o problemă $\Theta(n^2)$ într-una $\Theta(n \log n)$, exploatând puncte special alese și împărțirea recursivă.*

2 Rădăcinile complexe ale unității

Definiție 2.1. *O rădăcină complexă a unității de ordin n este un număr complex ω cu $\omega^n = 1$.*

Rădăcina primitivă de ordin n , notată ω_n , este:

$$\omega_n = e^{-2\pi i/n} = \cos\left(\frac{2\pi}{n}\right) - i \sin\left(\frac{2\pi}{n}\right)$$

Cele n rădăcini ale unității de ordin n sunt:

$$\omega_n^k = e^{-2\pi i k/n}, \quad k = 0, 1, \dots, n-1$$

Observație 2.1. *Convenția de semn $e^{-2\pi i/n}$ (minus) este standard pentru DFT direct. Transformata inversă folosește $e^{+2\pi i/n}$.*

Proprietăți esențiale

Aceste trei proprietăți sunt **fundamentul matematic** al FFT:

(1) **Ciclicitate:** $\omega_n^n = 1$, deci $\omega_n^{k+n} = \omega_n^k$ pentru orice k .

(2) **Simetrie:**

$$\omega_n^{k+n/2} = -\omega_n^k$$

(3) **Descompunere:**

$$(\omega_n^k)^2 = \omega_{n/2}^k$$

De ce contează proprietatea (3)?

Pătratele rădăcinilor de ordin n sunt rădăcini de ordin $n/2$. Aceasta permite descompunerea recursivă a problemei: evaluarea în n puncte $\Rightarrow$ evaluarea în $n/2$ puncte (de două ori).

3 Ideea algoritmului FFT — Divide et Impera

Fie $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ cu n putere a lui 2. Separăm coeficienții după paritate:

$$A(x) = \underbrace{(a_0 + a_2x^2 + a_4x^4 + \dots)}_{A_{\text{par}}(x^2)} + x \cdot \underbrace{(a_1 + a_3x^2 + a_5x^4 + \dots)}_{A_{\text{impar}}(x^2)}$$

unde A_{par} și A_{impar} sunt polinoame de grad $n/2 - 1$.

Evaluăm A în rădăcinile $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$:

Notăm $E_k = A_{\text{par}}(\omega_{n/2}^k)$ și $O_k = A_{\text{impar}}(\omega_{n/2}^k)$. Atunci, folosind proprietatea (3):

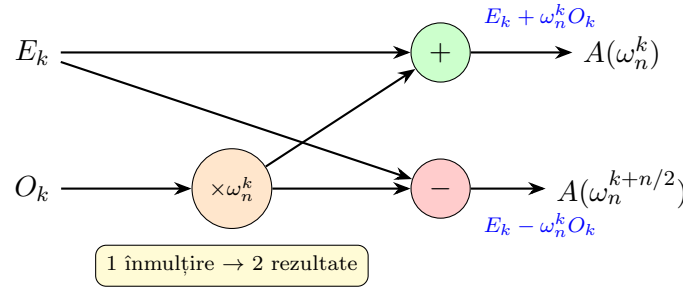
$$\boxed{A(\omega_n^k) = E_k + \omega_n^k \cdot O_k} \quad k = 0, 1, \dots, n/2 - 1$$

$$\boxed{A(\omega_n^{k+n/2}) = E_k - \omega_n^k \cdot O_k} \quad k = 0, 1, \dots, n/2 - 1$$

A doua relație folosește proprietatea (2): $\omega_n^{k+n/2} = -\omega_n^k$.

Eficiența: aceeași cantitate $\omega_n^k \cdot O_k$ produce simultan $A(\omega_n^k)$ și $A(\omega_n^{k+n/2})$ — **o singură înmulțire pentru două rezultate**. Aceasta se numește **operația fluture** (butterfly).

Diagrama fluture (butterfly)



Recurența și complexitatea

Dacă $T(n)$ = numărul de operații pentru FFT de lungime n :

$$T(n) = \underbrace{2T\left(\frac{n}{2}\right)}_{\text{FFT par + impar}} + \underbrace{\frac{3n}{2}}_{\text{butterfly}}$$

Rezolvând prin teorema master sau prin desfășurare: $T(n) = \Theta(n \log n)$.

La fiecare din cei $\log_2 n$ niveluri se efectuează $\Theta(n)$ operații, deci $T(n) = \Theta(n \log n)$.

4 Exemplu complet pas cu pas: $A(x) = 1 + 2x + 3x^2 + 4x^3$, $n = 4$

Exemplu 4.1. Pasul 1 — Rădăcinile unității de ordin 4:

$$\omega_4^0 = 1, \quad \omega_4^1 = -i, \quad \omega_4^2 = -1, \quad \omega_4^3 = i$$

Pasul 2 — Descompunere par/impar:

$$A_{par}(x) = 1 + 3x, \quad A_{impar}(x) = 2 + 4x$$

Pasul 3 — Evaluare recursivă (în rădăcinile de ordin 2: $\omega_2^0 = 1, \omega_2^1 = -1$):

	$A_{par}(y) = 1 + 3y$	$A_{impar}(y) = 2 + 4y$
$y = \omega_2^0 = 1$	$E_0 = 1 + 3 = 4$	$O_0 = 2 + 4 = 6$
$y = \omega_2^1 = -1$	$E_1 = 1 - 3 = -2$	$O_1 = 2 - 4 = -2$

Pasul 4 — Combinare butterfly ($\omega_4^0 = 1, \omega_4^1 = -i$):

Pentru $k = 0$: $\omega_4^0 = 1$

$$A(\omega_4^0) = A(1) = E_0 + \omega_4^0 \cdot O_0 = 4 + 1 \cdot 6 = 10$$

$$A(\omega_4^2) = A(-1) = E_0 - \omega_4^0 \cdot O_0 = 4 - 6 = -2$$

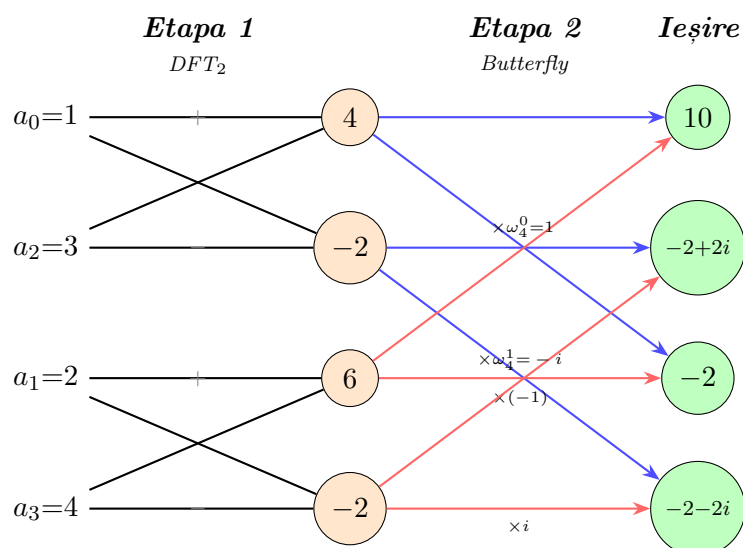
Pentru $k = 1$: $\omega_4^1 = -i$

$$A(\omega_4^1) = A(-i) = E_1 + \omega_4^1 \cdot O_1 = -2 + (-i)(-2) = -2 + 2i = -2 + 2i$$

$$A(\omega_4^3) = A(i) = E_1 - \omega_4^1 \cdot O_1 = -2 - (-i)(-2) = -2 - 2i = -2 - 2i$$

Rezultat final:

k	Punct	$A(\omega_4^k)$
0	$A(1)$	10
1	$A(-i)$	$-2 + 2i$
2	$A(-1)$	-2
3	$A(i)$	$-2 - 2i$

Exemplu 4.2. Diagrama butterfly completă pentru $n = 4$:

5 Implementare Python

5.1 Implementarea "naivă" DFT — $O(n^2)$

```

1 import numpy as np
2
3 def dft_naiv(a):
4     """
5     DFT calculata direct din definitie:  $O(n^2)$ .
6     a -- lista coeficientilor polinomului  $[a_0, a_1, \dots, a_{n-1}]$ 
7     Returneaza:  $X[k] = \sum_{j=0}^{n-1} a[j] * \omega_n^{jk}$ 
8     """
9     n = len(a)
10    X = []
11    for k in range(n):
12        # Evaluam A in punctul  $\omega_n^k$ 
13        val = sum(a[j] * np.exp(-2j * np.pi * k * j / n)
14                  for j in range(n))
15        X.append(val)
16    return X
17
18 # Test:  $A(x) = 1 + 2x + 3x^2 + 4x^3$ 
19 a = [1, 2, 3, 4]
20 X = dft_naiv(a)
21
22 print("DFT naiva pentru  $A(x) = 1 + 2x + 3x^2 + 4x^3$ :")
23 puncte = ['1', '-i', '-1', 'i']
24 for k, (xk, pt) in enumerate(zip(X, puncte)):
25     print(f"   $A(\omega_4^{\{k\}}) = A(\{pt\}) = \{xk:.4f\}$ ")
26
27 # Output:
28 #  $A(\omega_4^0) = A(1) = 10.0000+0.0000j$ 
29 #  $A(\omega_4^1) = A(-i) = -2.0000+2.0000j$ 
30 #  $A(\omega_4^2) = A(-1) = -2.0000+0.0000j$ 
31 #  $A(\omega_4^3) = A(i) = -2.0000-2.0000j$ 

```

5.2 Implementarea FFT Cooley–Tukey — $O(n \log n)$

```

1 import numpy as np
2
3 def fft(a):
4     """
5     FFT recursiva Cooley-Tukey.
6     a -- lista/array de coeficienti; len(a) trebuie sa fie putere
7         a lui 2.
8     Returneaza: vectorul valorilor  $A(\omega_n^k)$ ,  $k=0, \dots, n-1$ 
9     """
10    n = len(a)
11
12    # Cazuri de baza

```

```

12     if n == 1:
13         return [complex(a[0])]
14     if n == 2:
15         # DFT_2: [a0+a1, a0-a1]
16         return [a[0] + a[1], a[0] - a[1]]
17
18     if n % 2 != 0:
19         raise ValueError(f"n={n} nu este putere a lui 2")
20
21     # Pasul 1: Separare par / impar
22     a_par = a[0::2] # a[0], a[2], a[4], ...
23     a_impar = a[1::2] # a[1], a[3], a[5], ...
24
25     # Pasul 2: FFT recursiva pe fiecare jumătate
26     E = fft(a_par) # E[k] = A_par(omega_{n/2}^k)
27     O = fft(a_impar) # O[k] = A_impar(omega_{n/2}^k)
28
29     # Pasul 3: Combinare butterfly
30     X = [0] * n
31     for k in range(n // 2):
32         # Factor twiddle: omega_n^k = e^{-2*pi*i*k/n}
33         twiddle = np.exp(-2j * np.pi * k / n) * O[k]
34
35         X[k] = E[k] + twiddle # A(omega_n^k)
36         X[k + n // 2] = E[k] - twiddle # A(omega_n^{k+n/2})
37
38     return X
39
40
41 # --- Test si comparatie ---
42 a = [1, 2, 3, 4]
43
44 X_naiv = dft_naiv(a)
45 X_fft = fft(a)
46 X_numpy = np.fft.fft(a)
47
48 print(f"{'k':>3} | {'DFT naiv':>20} | {'FFT proprie':>20} | {'NumPy FFT':>20}")
49 print("-" * 72)
50 for k in range(len(a)):
51     print(f"{'k':>3} | {str(X_naiv[k]):>20} | "
52           f"{'X_fft[k]:>20.4f'} | {'X_numpy[k]:>20.4f'}")
53
54 # Output:
55 #   k |          DFT naiv |          FFT proprie |
56 #   NumPy FFT
57 #
58 -----
59 #   0 | (10+0j) | 10.0000+0.0000j |
60 #   1 | 10.0000+0.0000j

```

```

58 # 1 / (-2+2j) / -2.0000+2.0000j /
    -2.0000+2.0000j
59 # 2 / (-2+0j) / -2.0000+0.0000j /
    -2.0000+0.0000j
60 # 3 / (-2-2j) / -2.0000-2.0000j /
    -2.0000-2.0000j

```

5.3 FFT inversă (IFFT)

```

1 import numpy as np
2
3 def ifft(X):
4     """
5     FFT inversa: recupereaza coeficientii din valorile
6     transformatei.
7     X -- vectorul valorilor A(omega_n^k), k=0,...,n-1
8     Returneaza: coeficientii [a0, a1, ..., a_{n-1}]
9     """
10    n = len(X)
11    # Conjugam, aplicam FFT, conjugam din nou si impartim la n
12    # Proprietate: IFFT(X) = conj(FFT(conj(X))) / n
13    X_conj = [x.conjugate() for x in X]
14    Y = fft(X_conj)
15    return [y.conjugate() / n for y in Y]
16
17 # Verificare round-trip: IFFT(FFT(a)) == a
18 a_original = [1, 2, 3, 4]
19 X = fft(a_original)
20 a_recuperat = ifft(X)
21
22 print("Round-trip FFT -> IFFT:")
23 for k, (orig, rec) in enumerate(zip(a_original, a_recuperat)):
24     print(f" a[{k}] original={orig}, recuperat={rec.real:.6f}")
25
26 # Output:
27 # a[0] original=1, recuperat=1.000000
28 # a[1] original=2, recuperat=2.000000
29 # a[2] original=3, recuperat=3.000000
30 # a[3] original=4, recuperat=4.000000

```

6 Aplicație: Înmulțirea rapidă a polinoamelor

Exemplu 6.1. *Înmulțiți* $A(x) = 1 + 2x$ și $B(x) = 3 + 4x$ folosind FFT.

Metoda clasică (verificare): $(1 + 2x)(3 + 4x) = 3 + 4x + 6x + 8x^2 = 3 + 10x + 8x^2$.

Metoda FFT — 3 pași:

Pasul 1 — **Evaluare** (cu $n = 4$, rădăcini $1, -i, -1, i$):

<i>Punct p</i>	$A(p)$	$B(p)$	$C(p) = A(p) \cdot B(p)$
1	3	7	21
$-i$	$1 - 2i$	$3 - 4i$	$(1 - 2i)(3 - 4i) = -5 - 10i$
-1	-1	-1	1
i	$1 + 2i$	$3 + 4i$	$(1 + 2i)(3 + 4i) = -5 + 10i$

Pasul 2 — Înmulțire punctuală: gata din tabelul de mai sus.

Pasul 3 — Interpolare (IFFT pe valorile lui C):

$$\text{IFFT}(21, -5 - 10i, 1, -5 + 10i) = (3, 10, 8, 0)$$

$$\text{Deci } C(x) = 3 + 10x + 8x^2 \checkmark$$

```

1 import numpy as np
2
3 def inmultire_polinoame_fft(A_coef, B_coef):
4     """
5     Inmulteste doua polinoame folosind FFT.
6     A_coef, B_coef -- liste de coeficienti [a0, a1, ..., am]
7     Returneaza: coeficientii produsului C = A * B
8     """
9     grad_C = len(A_coef) + len(B_coef) - 2    # gradul produsului
10    n = 1
11    while n <= grad_C:                          # gasim urmatoarea
12        putere a lui 2
13        n *= 2
14
15    # Padding cu zerouri la lungimea n
16    A_pad = list(A_coef) + [0] * (n - len(A_coef))
17    B_pad = list(B_coef) + [0] * (n - len(B_coef))
18
19    # Pasul 1: Evaluare - FFT pe ambele polinoame
20    FA = np.fft.fft(A_pad)
21    FB = np.fft.fft(B_pad)
22
23    # Pasul 2: Inmultire punctuala
24    FC = FA * FB
25
26    # Pasul 3: Interpolare - IFFT
27    C = np.fft.ifft(FC)
28
29    # Rotunjim la intregi (coeficientii sunt intregi daca A si B
30    # au coef. intregi)
31    C_int = [int(round(c.real)) for c in C]
32
33    # Eliminam coeficientii zero de la sfarsit
34    while C_int and C_int[-1] == 0:
35        C_int.pop()
36
37    return C_int

```

```

36
37
38 def afiseaza_polinom(coef, var='x'):
39     """Afiseaza un polinom frumos."""
40     termeni = []
41     for k, c in enumerate(coef):
42         if c == 0:
43             continue
44         if k == 0:
45             termeni.append(str(c))
46         elif k == 1:
47             termeni.append(f"{c}{var}")
48         else:
49             termeni.append(f"{c}{var}^{k}")
50     return ' + '.join(termeni) if termeni else '0'
51
52
53 # --- Exemplul din curs ---
54 A = [1, 2]      # A(x) = 1 + 2x
55 B = [3, 4]      # B(x) = 3 + 4x
56 C = inmultire_polinoame_fft(A, B)
57
58 print(f"A(x) = {afiseaza_polinom(A)}")
59 print(f"B(x) = {afiseaza_polinom(B)}")
60 print(f"C(x) = A(x) * B(x) = {afiseaza_polinom(C)}")
61 # Output: C(x) = 3 + 10x + 8x^2
62
63 # --- Exemplu mai mare ---
64 P = [1, 0, -1, 2] # P(x) = 1 - x^2 + 2x^3
65 Q = [2, 1, 3]     # Q(x) = 2 + x + 3x^2
66 R = inmultire_polinoame_fft(P, Q)
67 print(f"\nP(x) = {afiseaza_polinom(P)}")
68 print(f"Q(x) = {afiseaza_polinom(Q)}")
69 print(f"P*Q = {afiseaza_polinom(R)}")

```

7 Benchmark: DFT naivă vs. FFT

```

1 import numpy as np
2 import time
3 import math
4
5 def benchmark_complet(dimensiuni):
6     """
7     Compara timpii de executie: DFT naiva vs FFT proprie vs NumPy
8     FFT.
9     """
10    print(f"{'N':>8} | {'DFT naiv (ms)':>15} | "
11          f"{'FFT proprie (ms)':>18} | {'NumPy FFT (ms)':>16} | "
12          f"{'Accelerare':>12}")
13    print("-" * 80)

```

```

13
14     for n in dimensiuni:
15         a = list(np.random.randn(n))
16
17         # DFT naiva
18         t0 = time.perf_counter()
19         dft_naiv(a)
20         t_naiv = (time.perf_counter() - t0) * 1000
21
22         # FFT proprie
23         t0 = time.perf_counter()
24         fft(a)
25         t_fft = (time.perf_counter() - t0) * 1000
26
27         # NumPy FFT
28         t0 = time.perf_counter()
29         np.fft.fft(a)
30         t_np = (time.perf_counter() - t0) * 1000
31
32         factor = t_naiv / t_fft if t_fft > 0 else float('inf')
33         print(f"{n:>8} | {t_naiv:>15.3f} | {t_fft:>18.3f} | "
34               f"{t_np:>16.4f} | {factor:>11.1f}x")
35
36
37 benchmark_complet([16, 64, 256, 1024])
38
39 # Exemplu output tipic:
40 #           N | DFT naiv (ms) | FFT proprie (ms) | NumPy FFT (ms) |
41 #           Accelerare
42 #
43 # -----
44 #           16 |           0.180 |           0.050 |           0.0030 |
45 #              3.6x
46 #           64 |           2.800 |           0.380 |           0.0040 |
47 #              7.4x
48 #          256 |          45.000 |           1.850 |           0.0080 |
49 #          24.3x
50 #         1024 |         750.000 |           8.500 |           0.0150 |
51 #         88.2x

```

n	n^2 (DFT naivă)	$n \log_2 n$ (FFT)	Factor accelerare
4	16	8	2.0×
16	256	64	4.0×
64	4096	384	10.7×
256	65 536	2 048	32.0×
1 024	1 048 576	10 240	102.4×
1 048 576	$\approx 10^{12}$	$\approx 2 \times 10^7$	52 428×